Documento di progettazione

MyLibrary

ALESSANDRO SABIA

Ingegneria del Software
2026/2027

Documento di progettazione 0
1. Introduzione e Modello Dati 2
2. Architettura del Software e Package 3
    2.1 Livello di Dominio (Model) 4
    2.2 Livello Service e Persistence 5
    2.3 Livello di Presentazione (UI) 6
3. Diagrammi di Sequenza 6
    3.1 Registrazione di un nuovo prestito 7
    3.2 Flusso di Persistenza (Caricamento dell'archivio) 8
4. Coesione 9
5. Accoppiamento 10
6. Principi di buona progettazione 11
7. Ambito della release 1 12
    7.1 Tracciabilità verso l'SRS 12
    7.2 Scelte progettuali e trade-off 14
    7.3 Possibili evoluzioni 14
8. Tracciabilità Requisiti 15

1. Introduzione e Modello Dati

Il presente documento descrive l'architettura e la progettazione della Release 1 per il sistema software di una biblioteca Universitaria. La release 1 copre i flussi essenziali dei requisiti a priorità alta, per alcuni di essi realizza solo la parte necessaria al ciclo di vita base (rimando alla sezione 7).

Di conseguenza, il modello dati è stato snellito per gestire unicamente le informazioni fondamentali per questa release:

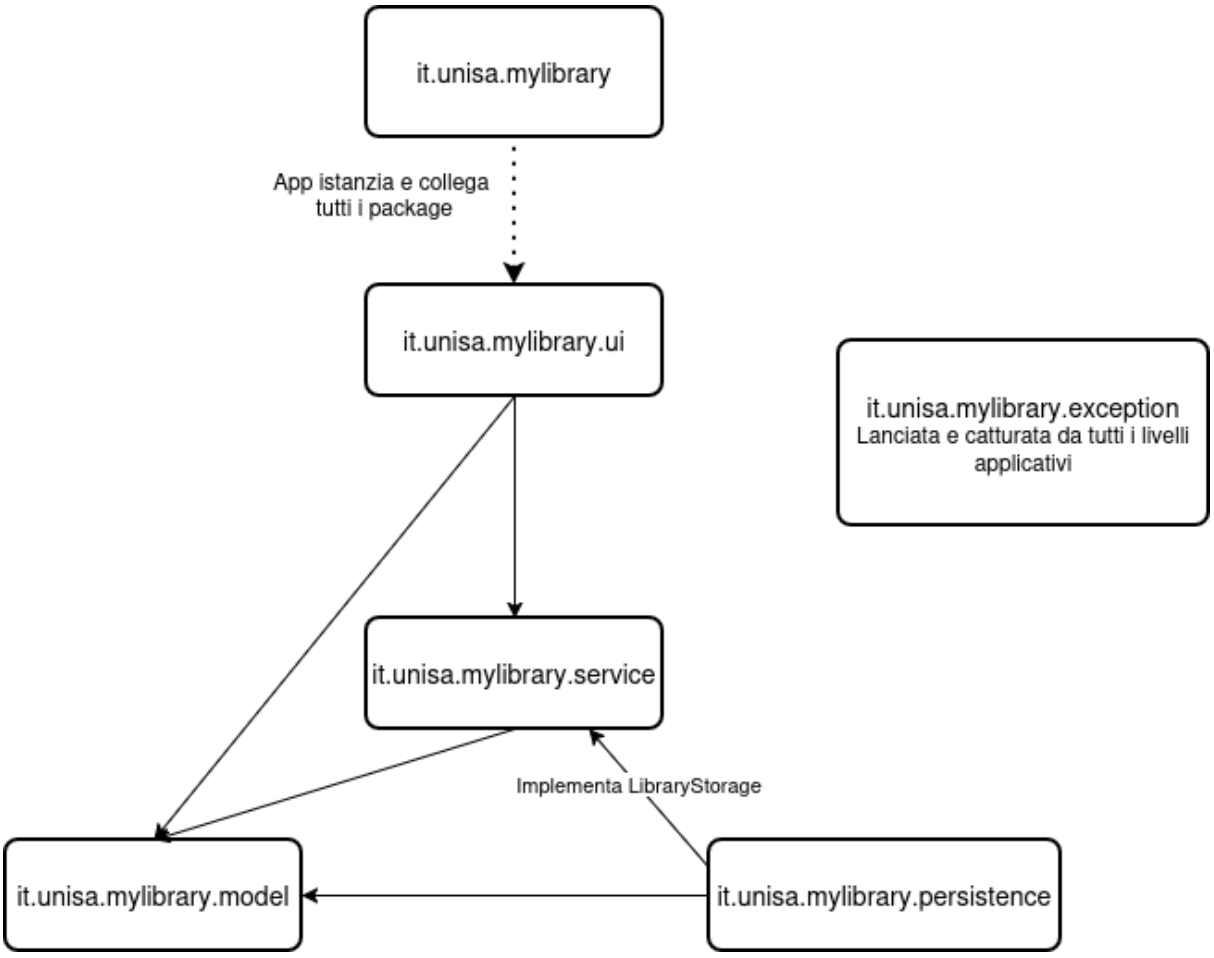
- **Libro:** Ogni volume è definito in modo univoco da 5 campi essenziali: codice ISBN, titolo, lista degli autori, anno di pubblicazione e numero di copie totali fisicamente acquisite. I campi di classificazione facoltativi (tipologia e genere letterario) non sono previsti in questa release.
- **Lettore:** L'anagrafica utente è tracciata tramite 4 campi obbligatori: nome, cognome, matricola istituzionale (identificatore univoco) ed e-mail. Sono state escluse le informazioni accessorie (ruolo, numero di telefono) e non sono presenti i meccanismi logici complessi relativi all'indice di affidabilità o al blocco operativo automatico del lettore.
- **Prestito:** L'entità che traccia l'erogazione collega in modo diretto la matricola del lettore e l'ISBN del libro, registrando esclusivamente la data di inizio, la data di scadenza prevista e l'effettiva data di restituzione. Non è prevista la registrazione di note testuali o dello stato conservativo del volume (usura).

Tutte le scelte di scoping e i requisiti rimandati a versioni successive rispetto alle specifiche ideali dell'SRS sono mappati e giustificati nella successiva Sezione 7.

2. Architettura del Software e Package

L'applicazione adotta un'architettura a livelli con una netta separazione delle responsabilità:

- **model**: Cuore dell'applicazione. Contiene le entità (Book, Reader, Loan) e l'Archive (stato globale). Non ha alcuna dipendenza verso gli altri layer.
- **persistence**: Incapsula l'I/O su file system. Fornisce l'implementazione concreta (ArchiveStorage) dell'astrazione definita dai servizi.
- **service**: Gestisce la validazione, le regole di business e definisce l'astrazione LibraryStorage. Interagisce con la persistenza applicando in modo puro il DIP.
- **exception**: Centralizza la gestione degli errori con un'unica classe (LibraryException).
- **ui**: Livello di presentazione JavaFX dedicato esclusivamente all'interfaccia utente.
- **root (it.unisa.mylibrary)**: Contiene la classe Main (App) che gestisce l'avvio: istanzia modello, storage e servizi, passandoli alle viste.



Note architetturali sulla progettazione: La struttura garantisce alta coesione e basso accoppiamento. I Service rappresentano l'unica vera autorità per la convalida delle regole di business; la UI ne replica alcune visivamente (es. limite di 3 slot prestiti o blocco tasti per volumi esauriti) solo a scopo preventivo e di user experience. L'uso dell'interfaccia LibraryStorage disaccoppia i servizi dall'implementazione dell'I/O, facilitando il testing isolato.Infine, l'aver centralizzato la creazione dei servizi e delle viste principali nella classe App evita costrutti sparsi e dipendenze dirette all'interno delle classi grafiche.

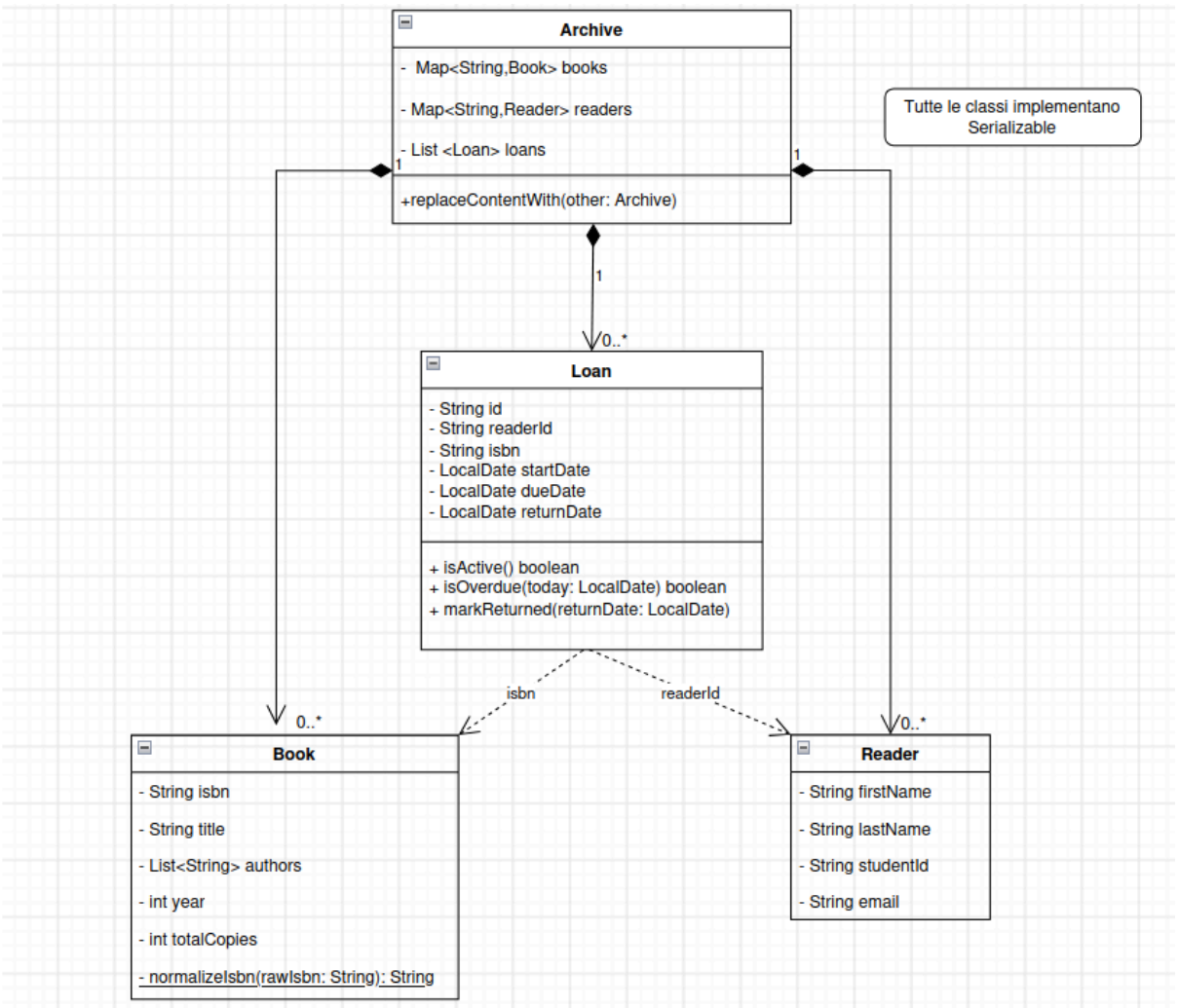
Per massimizzare la leggibilità, il diagramma globale dell'architettura è stato suddiviso in tre livelli fondamentali: il Modello di Dominio, il comparto Service/Persistence e il livello di Presentazione (UI).

2.1 Livello di Dominio (Model)

Il modello è il cuore dell'applicazione e detiene lo stato globale. Non ha dipendenze esterne. Tutte le entità del dominio implementano l'interfaccia Serializable, requisito necessario per consentire il salvataggio su file tramite ArchiveStorage.

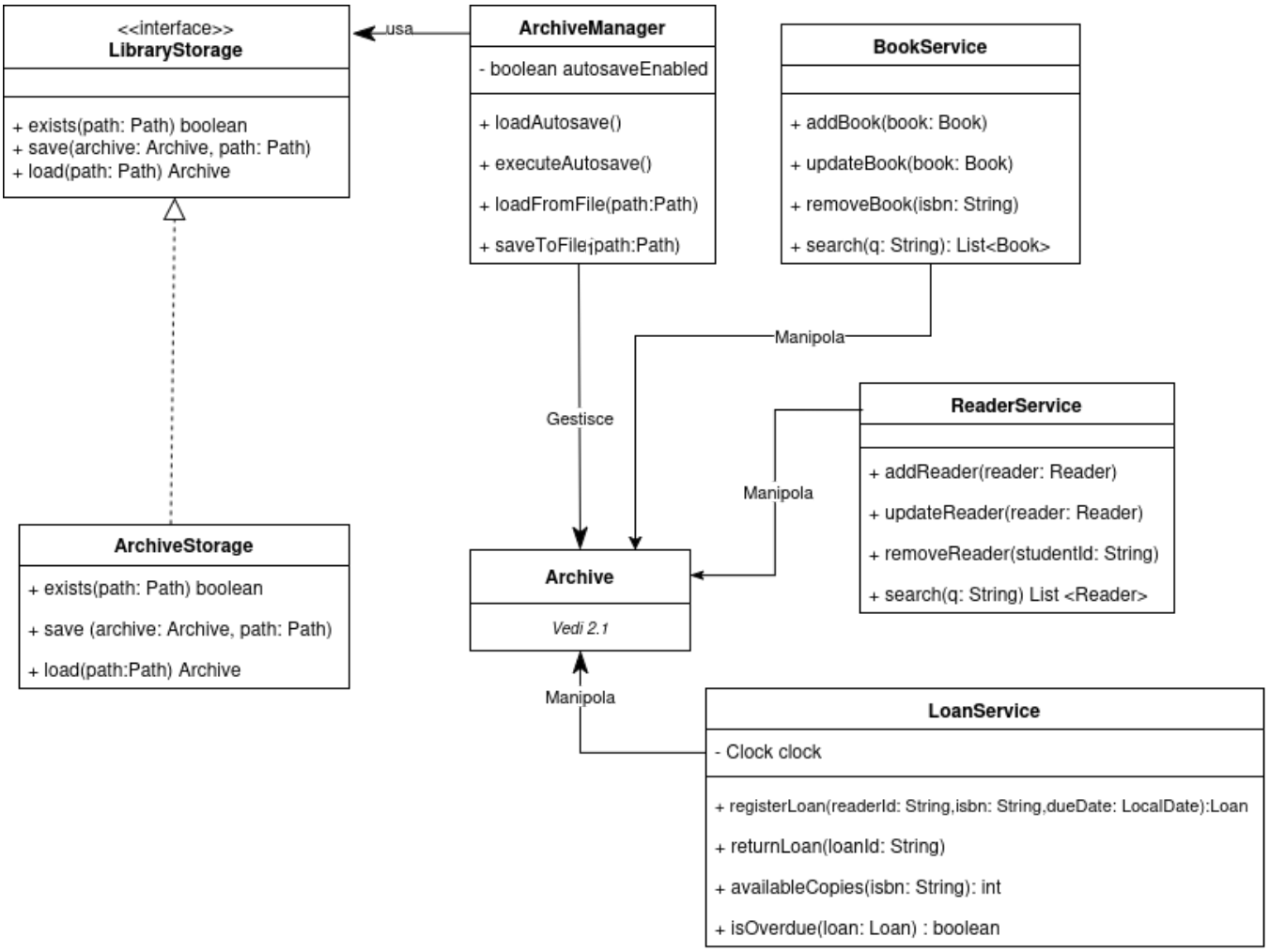
Per prevenire problemi di riferimenti circolari e semplificare la serializzazione, la classe Loan referenzia Reader e Book tramite i loro identificativi testuali (readerId e isbn) anziché utilizzare riferimenti diretti agli oggetti. L'integrità di tali riferimenti è garantita a monte dai Service.

Nota ai diagrammi: Per brevità visiva sono stati omessi i metodi getter e setter. Si specifica inoltre che i metodi equals e hashCode di ogni entità sono basati univocamente sul proprio identificatore primario (ISBN per Book, matricola per Reader, e un ID generato automaticamente per Loan).



2.2 Livello Service e Persistence

I servizi contengono la logica applicativa e le regole di business (RD). Gestiscono le entità del modello e delegano la persistenza sfruttando il principio di Inversione delle Dipendenze (DIP).

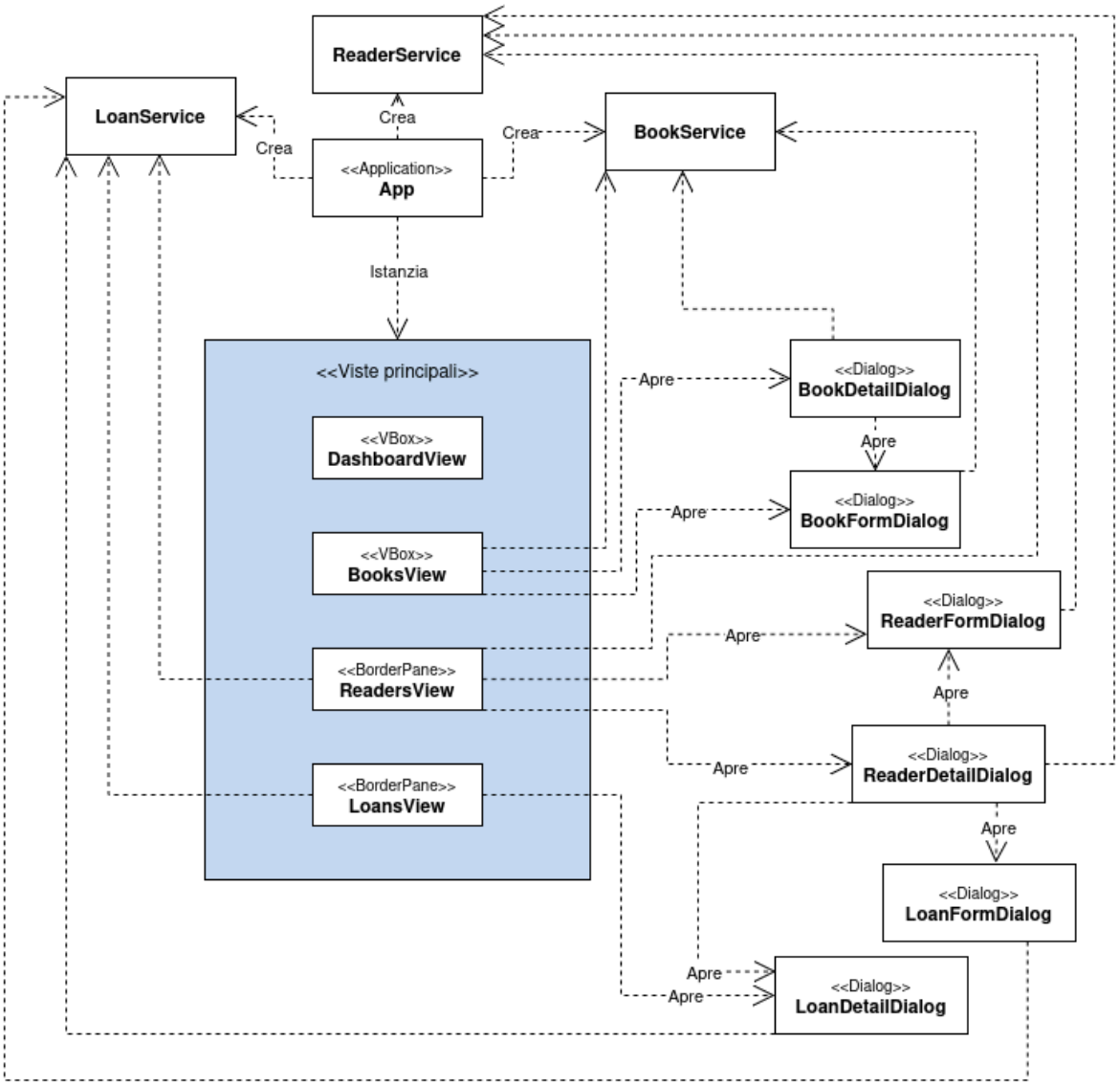


2.3 Livello di Presentazione (UI)

L'architettura adottata per l'interfaccia si basa su un approccio snello senza l'utilizzo di classi Controller separate. Le classi UI estendono direttamente i contenitori nativi di JavaFX (VBox, BorderPane, Dialog) e gestiscono localmente gli eventi grafici.

Questa scelta riduce il codice strutturale pur mantenendo una netta separazione dei livelli: le Viste non contengono logica di business, ma agiscono da intermediari, delegando validazioni ed elaborazioni ai rispettivi Service (le uniche eccezioni sono i controlli preventivi descritti in sezione 2).

La classe radice App funge da punto d'avvio: istanzia i Service, crea le Viste e passa loro i riferimenti necessari tramite costruttore. Le funzionalità grafiche ricorrenti (come i pop-up di errore o conferma) sono invece centralizzate nelle classi di utilità UiUtils e Alerts.



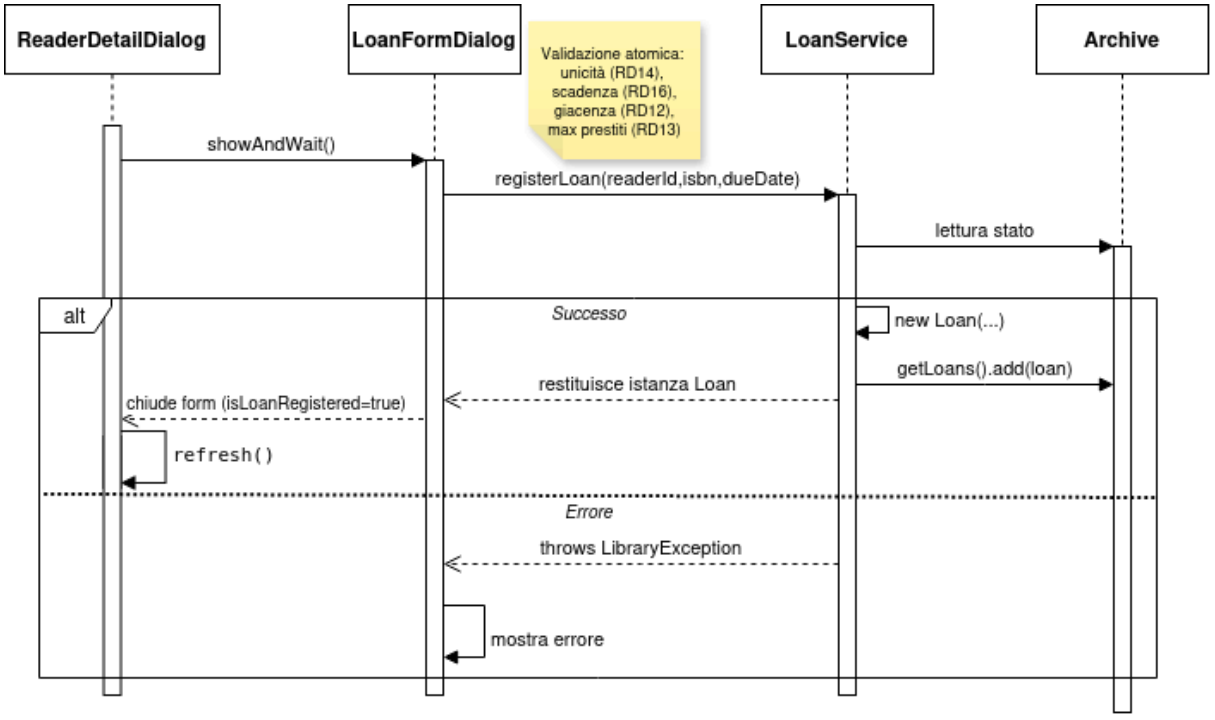
3. Diagrammi di Sequenza

I seguenti diagrammi illustrano le interazioni tra i livelli UI, Service, Model e Persistence per le due operazioni più significative del sistema.

3.1 Registrazione di un nuovo prestito

Il diagramma illustra il flusso di erogazione, evidenziando due aspetti chiave:

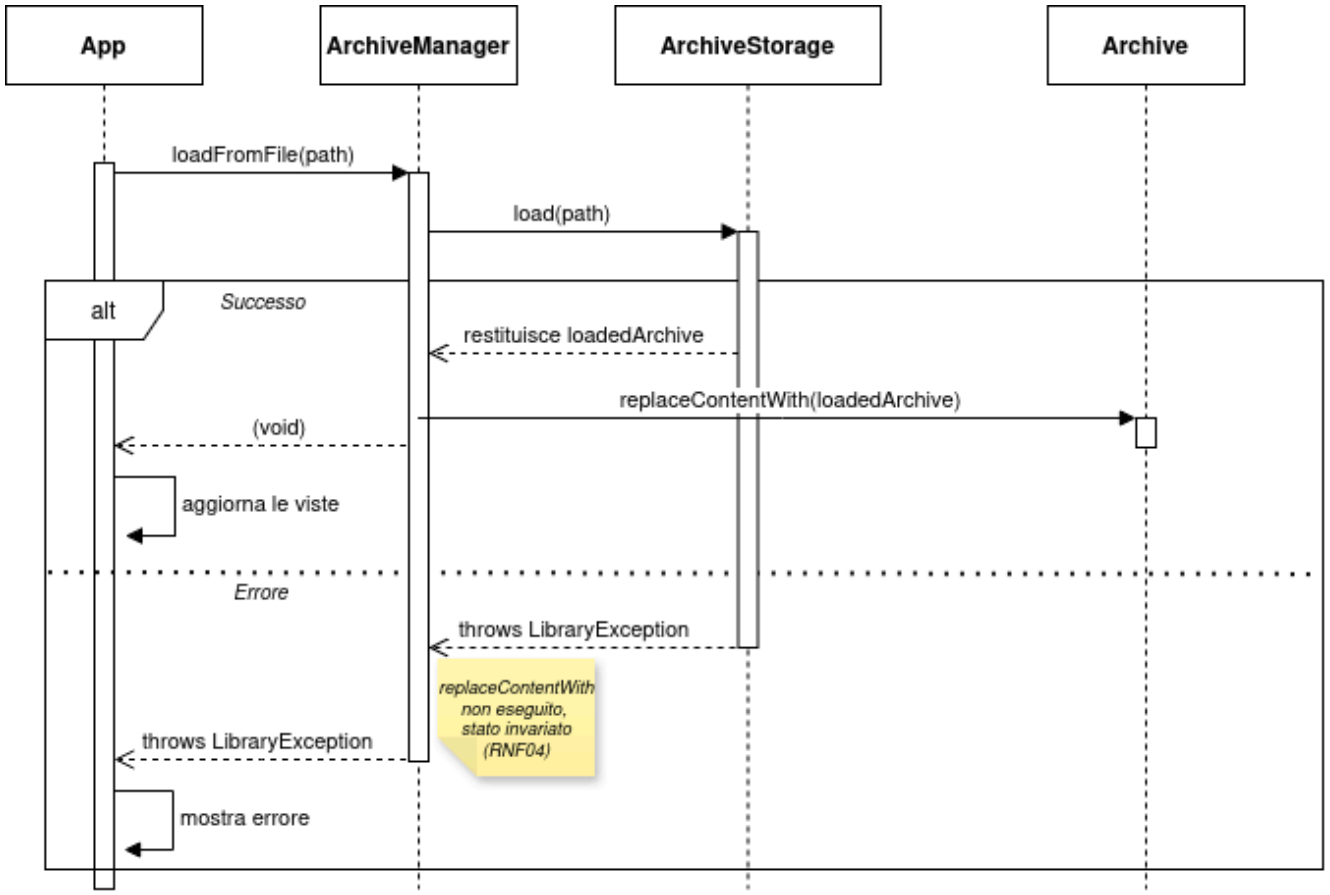
1. **Delegazione e Refresh:** Il chiamante (ReaderDetailDialog), una volta delegata l'operazione al form e verificato il successo, aggiorna autonomamente l'interfaccia.
2. **Gestione degli Errori:** Mentre alcune violazioni sono prevenute visivamente (es. max prestiti tramite slot fissi o disabilitazione dei libri esauriti), altre anomalie in fase di input (es. RD14 o RD16) scatenano un'Eccezione di Dominio. Il form la intercetta, mostra un avviso e interrompe la chiusura della finestra, permettendo all'operatore di correggere i dati senza perdere il contesto.



3.2 Flusso di Persistenza (Caricamento dell'archivio)

Il diagramma illustra il disaccoppiamento tra interfaccia (App), logica applicativa (ArchiveManager) e File System, mediato dall'astrazione LibraryStorage. I punti salienti sono:

1. **Gestione I/O nella UI:** L'acquisizione del file tramite FileChooser e il prompt di conferma (per evitare sovrascritture accidentali) sono risolti interamente nella UI prima di invocare il manager.
2. **Integrità dello Stato (RNF04):** In caso di errore di lettura o file corrotto, l'eccezione blocca il flusso prima di invocare replaceContentWith. L'errore viene propagato alla UI senza arrestare l'applicazione, garantendo che i dati in memoria rimangano intatti.



4. Coesione

La seguente tabella analizza il grado in cui le responsabilità all'interno di ciascuna classe sono focalizzate e correlate.

Classe / Modulo	Livello di Coesione	Descrizione
Book, Reader, Loan	Funzionale (Informativa)	Raggruppano esclusivamente gli attributi di stato e i relativi comportamenti base (es. isActive()) di una singola entità del dominio.
Archive	Funzionale	Raggruppa unicamente le collezioni che costituiscono lo stato globale della biblioteca.
ArchiveStorage	Funzionale	Concentrata su un unico compito: serializzazione e deserializzazione dell'intero stato su file system.
LibraryException	Funzionale	Ha l'unico scopo di incapsulare e propagare gli errori legati alle violazioni di dominio.
ArchiveManager	Funzionale	Si dedica alla mediazione tra il modello e il sottosistema di I/O (caricamento, salvataggio e autosalvataggio).
BookService, ReaderService, LoanService	Funzionale	Ogni servizio offre funzionalità mirate esclusivamente alla validazione formale e al ciclo di vita di una singola entità.
Classi View e Dialog	Comunicazionale	I metodi interni operano tutti sul medesimo set di dati (le proprietà grafiche e l'entità associata) per aggiornare l'interfaccia.
Alerts	Funzionale	Raggruppa unicamente i metodi per istanziare e mostrare a schermo i popup (errori, conferme).
UiUtils	Logica	Raggruppa funzioni di utilità statica eterogenee. Nota: il metodo executeReturnLoan raccoglie volutamente in un'unica procedura il flusso condiviso di conferma e restituzione, centralizzando il cordinamento tra UI e Service ed evitando duplicazioni nelle varie Viste chiamanti.
App (Classe Main)	Temporale / Procedurale	Istruzioni raggruppate perché eseguite in una specifica sequenza di avvio (creazione oggetti, passaggio parametri, setup UI).

5. Accoppiamento

La tabella seguente descrive le relazioni tra i componenti e la tipologia di dipendenza

Relazione	Livello di Accoppiamento	Descrizione
ArchiveManager , LibraryStorage	Per Timbro	Il manager passa l'intera istanza di Archive all'astrazione LibraryStorage per l'I/O, senza conoscere i dettagli del file system.
Service , Archive	Per Timbro / Area Comune	Tutti i servizi ricevono la medesima istanza dell'archivio, operando sullo stato mutabile delle collezioni condivise.
Service , Model	Per Timbro	I Service ricevono e restituiscono istanze complete delle entità
App ,Viste / Servizi	Per Timbro	In qualità di classe di avvio, App presenta un alto fan-out fisiologico poiché istanzia e passa i riferimenti necessari ai costruttori di quasi tutte le viste.
Viste , Servizi	Per dati	La vista padre richiede nel costruttore alcuni servizi che non utilizza direttamente, ma che è costretta a mantenere solo per poterli passare alle finestre di dialogo figlie al momento della loro apertura.
LoanFormDialog , LoanService	Per Dati	L'interfaccia raccoglie input primitivi (ID utente, ISBN, data) e li passa al servizio per avviare la validazione e creazione del prestito.

6. Principi di buona progettazione

Il progetto è stato strutturato applicando i principi S.O.L.I.D. e le buone pratiche di ingegneria del software.

- **Dependency Inversion Principle (DIP)**: Applicato per isolare la logica applicativa dall'infrastruttura di I/O.L'interfaccia LibraryStorage è definita direttamente dal livello Service: in questo modo il layer di persistenza dipende dal layer applicativo e non viceversa ,garantendo un'Inversione delle Dipendenze pura.
- **Open/Closed Principle (OCP)**: L'architettura di persistenza è aperta all'estensione ma chiusa alla modifica. È possibile introdurre nuovi formati o tecnologie semplicemente creando una nuova classe che implementi LibraryStorage, garantendo la sostituibilità del supporto di persistenza senza dover toccare il codice di ArchiveManager (salvo la classe Main App, che resta l'unica preposta a scegliere la specifica implementazione).
- **Liskov Substitution Principle (LSP)**: La correttezza dell'astrazione dello storage garantisce che sia possibile sostituire l'implementazione concreta legata al file system (ArchiveStorage) con un'altra implementazione senza che il comportamento atteso del manager venga alterato.
- **Interface Segregation Principle (ISP)**: Pur essendoci una sola interfaccia nel progetto (LibraryStorage), quest'ultima è estremamente coesa, piccola e snella. Espone esclusivamente i tre metodi strettamente necessari al manager (exists, save, load).
- **Riduzione dell'Accoppiamento (Passaggio esplicito delle dipendenze)**: L'accoppiamento strutturale è stato limitato evitando l'istanziazione diretta dei servizi di business all'interno delle Viste. I riferimenti operativi vengono creati una sola volta all'avvio e passati dall'esterno tramite i costruttori.

  Nota architetturale: Fanno eccezione a questa regola le classi di supporto Alerts e UiUtils, che sono strutturate con metodi statici per convenienza d'uso globale.

- **Single Responsibility Principle (SRP) e Separation of Concerns**: Esiste una chiara distinzione delle responsabilità. I componenti visivi gestiscono l'input/output grafico, delegando prevalentemente le logiche di dominio e le validazioni ai Service.
- **DRY (Don't Repeat Yourself)**: Il principio è applicato in più punti. Un esempio architetturale rilevante è la registrazione della restituzione: l'azione è innescabile da tre punti di ingresso grafici distinti, ma il flusso converge centralmente in un'unica procedura di supporto (UiUtils.executeReturnLoan), che gestisce il double-check di sicurezza e l'orchestrazione col Service. Allo stesso modo, le maschere di inserimento dati condividono il layout cambiando comportamento tramite flag.
- **Flessibilità delle Regole di Dominio**: La validazione dell'e-mail controlla la correttezza formale ma non impone il vincolo esclusivo sul dominio @studenti.unisa.it. Scelta voluta per consentire l'iscrizione a docenti o personale esterno.

7. Ambito della release 1

Questa sezione definisce il perimetro d'azione della Release 1. Al fine di garantire un rilascio stabile dei flussi critici, la progettazione si concentra sui requisiti a priorità Alta definiti nel documento SRS (per alcuni di essi realizza solo la parte necessaria al ciclo di vita base), rimandando le funzionalità a priorità Media o Bassa alle release successive.

7.1 Tracciabilità verso l'SRS

La seguente tabella mappa le scelte di perimetro:

Requisiti Coinvolti	Descrizione Breve	Release 1 / Rimandato	Motivazione / Note Progettuali
RD15	Conservazione storico prestiti	Release 1	Pur essendo a priorità Bassa nell'SRS, è stato incluso nel modello per coerenza con il ciclo di vita base del dominio.
IU10b	Storico prestiti e statistiche lettore	Rimandato (Statistiche)	Le statistiche aggregate, pur avendo priorità Alta, richiedono un'elaborazione dati complessa. Solo lo storico base è in Release 1.
ED04	Dati del lettore	Rimandato (Campi facoltativi)	Il modello dati supporta solo i campi obbligatori (priorità Alta). Ruolo e telefono sono rimandati per semplificare l'anagrafica.
IU19, IU20, IU21	Avvisi di sicurezza e abbandono form	Rimandato (Loan form)	Requisiti a priorità Media. Progettati e previsti sui form principali (Book e Reader), sono rimandati per la maschera dei prestiti.
IU16, IU16a	Finestra di restituzione prestito	Rimandato (Finestra dedicata)	Essendo lo stato conservativo e le note fuori ambito per la Release 1 (vedi sotto), la restituzione richiede la modifica di un solo flag. A livello di design

Requisiti Coinvolti	Descrizione Breve	Release 1 / Rimandato	Motivazione / Note Progettuali
			si è scelto un unico passaggio di conferma (Alert), rendendo ridondante lo sviluppo di una vista dedicata (requisito a priorità Alta).
IU09, RF03	Automazione inserimento copie e copie incrementali	Rimandato	Sebbene RF03 (Inserimento) abbia priorità Alta, la gestione autocompilante incrementale via ISBN (IU09, priorità Bassa) impone logiche UI complesse. Per la Release 1, l'aumento delle copie si ottiene modificando manualmente l'attributo dal form del libro.
IU15	Modalità visualizzazione prestiti	Rimandato	Requisito a priorità Media riguardante un toggle puramente visivo, non bloccante.
RF11, RD08-11, IU11b, IU12	Affidabilità e blocco operativo	Rimandato	Modulo avanzato a priorità Bassa.
RF24, RD17, IU16e, IU23	Gestione smarrimento volumi	Rimandato	Funzionalità accessoria e di contorno a priorità Bassa.
ED01b, RD05, RF07, RF19, RF20, IU07	Classificazione e filtri avanzati	Rimandato	Attributi facoltativi (tipologia, genere) a priorità Bassa.
RF22, ED05b, RD18, RD19, IU10c, IU14b, IU16b-d	Note e stato conservativo (usura)	Rimandato	Tracciamento avanzato a priorità Bassa, escluso dal modello per la Release 1.

7.2 Scelte progettuali e trade-off

Il design del sistema si basa su compromessi architetturali (trade-off) precisi, scelti per favorire la velocità di esecuzione e la semplicità del modello dati nella prima iterazione:

- **Copie come contatore:** I volumi sono trattati come aggregazioni numeriche e non come singoli esemplari fisici dotati di matricola univoca. Questo semplifica notevolmente l'associazione prestito-libro e il calcolo delle disponibilità.
- **Anno di Edizione:** L'attributo "Anno" fa riferimento all'anno dell'edizione fisica posseduta, non alla prima pubblicazione assoluta.
- **Archive come contenitore globale:** L'oggetto radice Archive non incapsula difensivamente il proprio stato, ma espone all'esterno i riferimenti diretti alle collezioni di dati (books, readers, loans). Questa scelta favorisce un accoppiamento fluido con il livello di persistenza.
- **Entità come contenitori di stato:** Le logiche di validazione e le regole di business sono demandate esplicitamente ai moduli Service. Di conseguenza, le entità del dominio (Book, Reader, Loan) operano prevalentemente come contenitori di stato.
- **Calcolo "Volumi a catalogo":** La metrica esposta nella Dashboard si ottiene sommando dinamicamente le copie fisiche totali di tutti i libri registrati, e non calcolando i titoli univoci immessi.

7.3 Possibili evoluzioni

Per supportare la scalabilità e le funzionalità delle release successive, l'architettura è predisposta per accogliere i seguenti miglioramenti:

- **Pattern Repository:** Introduzione di moduli di astrazione per mediare l'accesso ad Archive, slegando definitivamente i Service dalle collezioni in memoria e predisponendo il sistema per un futuro database relazionale.
- **Ordinamenti Polimorfici:** Adozione di interfacce polimorfiche nei Service per definire regole di ordinamento e filtraggio dinamico, limitando la proliferazione di metodi specifici per le ricerche.

8. Tracciabilità Requisiti

La seguente tabella mappa i requisiti (Funzionali, Dati, Dominio, UI e Non Funzionali) con le relative classi che ne implementano la logica o la presentazione

ID Requisito	Nome / Descrizione	Classi Coinvolte
Requisiti Funzionali (RF)		
RF01, RF02, RF08	Catalogo libri (visualizzazione, ordinamento, ricerca)	BooksView, BookService
RF03, RF05	Inserimento e Modifica libro	BookFormDialog, BookService

ID Requisito	Nome / Descrizione	Classi Coinvolte
RF04, RF06	Visualizzazione e Rimozione libro	BookDetailDialog, BookService
RF09, RF16	Lista lettori e Ricerca	ReadersView, ReaderService
RF10, RF14	Inserimento e Modifica lettore	ReaderFormDialog, ReaderService
RF12, RF13, RF15	Dettaglio, prestiti attivi e rimozione lettore	ReaderDetailDialog, ReaderService, LoanService
RF17, RF18	Visualizzazione e ordinamento globale prestiti	LoansView, LoanService
RF21	Registrazione prestito	LoanFormDialog, LoanService
RF23	Registrazione restituzione	LoanDetailDialog, UiUtils, LoanService
RF25	Visualizzazione bacheca (metriche e contatori)	DashboardView
RF26, RF27	Salvataggio e Caricamento	ArchiveManager, ArchiveStorage, App
Esigenze di Dati (ED)		
ED01a, ED04, ED05a	Dati obbligatori (Libro, Lettore, Prestito)	Book, Reader, Loan
ED02, ED03	Controlli di formato regex (ISBN, e-mail)	BookService, ReaderService
Regole di Dominio (RD)		
RD01, RD02, RD03	Vincoli giacenza, eliminazione e modifica libri	BookService, BookFormDialog
RD04a	Unicità del codice ISBN	BookService, Book
RD04b	Unicità della matricola	ReaderService
RD06	Calcolo dinamico giacenze	LoanService
RD07	Blocco eliminazione lettori con prestiti	ReaderService
RD12, RD13, RD14	Copie disponibili, max 3 prestiti, unicità volume	LoanService
RD15	Conservazione dello storico prestiti	ReaderDetailDialog
RD16	Durata e default scadenze del prestito	LoanService, LoanFormDialog
Interfaccia Utente (IU)		
IU01, IU02, IU03, IU04	GUI, Navigazione strutturale e menù persistente	App, DashboardView
IU05, IU06, IU08	Layout condiviso per form inserimento/modifica	BookFormDialog, ReaderFormDialog
IU10, IU10a, IU11, IU11a	Dettaglio lettore, slot prestiti e comandi base	ReaderDetailDialog

ID Requisito	Nome / Descrizione	Classi Coinvolte
IU10b	Storico prestiti e statistiche lettore (solo storico, senza statistiche)	ReaderDetailDialog
IU10c	Finestra di nuovo prestito (senza stato conservativo)	LoanFormDialog
IU13, IU13a, IU13b	Dettaglio libro, comandi e prestiti del volume	BookDetailDialog
IU14, IU14a, IU14c	Dettaglio prestito e innesco restituzione	LoanDetailDialog
IU16, IU16a	Finestra di restituzione e dati in sola lettura (solo conferma tramite Alert)	LoanDetailDialog, UiUtils, Alerts
IU17	Aggiornamento dinamico liste e viste	BooksView, ReadersView, LoansView, DashboardView
IU18	Evidenziazione cromatica ritardi	LoansView, BookDetailDialog, ReaderDetailDialog, UiUtils
IU19, IU20, IU21	Sicurezza scarto dati non salvati e abbandono form	BookFormDialog, ReaderFormDialog
IU22	Feedback operativo errori bloccanti	Alerts
Requisiti Non Funzionali (RNF)		
RNF01, RNF02, RNF07, RNF08	Prestazioni e Usabilità UI	Viste del livello ui
RNF03, RNF04	Integrità persistenza e gestione file non validi	ArchiveStorage, ArchiveManager
RNF05	Gestione globale delle eccezioni	App, LibraryException
RNF06	Atomicità operazioni	LoanService, ArchiveStorage
RNF09	Separazione dei livelli	Tutti i package
RNF10	Test automatici JUnit	Fase di implementazione e testing
RNF11, RNF12	JavaFX e Build con Maven	App, pom.xml